PROJECT DOCUMENTATION

LegalEase

AI Powered Legal Document Generator

Submitted by

Sakthi Sanjay(Team Leader)

Monisha

Praveen Kumar

Sandhiya

Raviraj

Department: B.Sc Computer Science

College: Prof Dhanapalan college of Science & Management

Academic Year: 2025-2028

LegalEase: AI-Powered Legal Document Generator

Project Description:

LegalEase leverages state-of-the-art Generative AI to simplify the creation of legal documents by providing customizable, accurate, and editable templates for a wide range of use cases. Users can generate employment contracts, lease agreements, NDAs, and more—all tailored to their specific inputs like involved parties, effective dates, and key terms.

Built with FastAPI and integrated with a powerful AI core, LegalEase ensures users receive professional-grade legal documents that maintain formatting standards, include custom branding (such as logos and fonts), and support features like editable previews and automatic term tables. With secure handling of sensitive user data and seamless document export options (.PDF, .DOCX, .TXT), the platform bridges the gap between accessibility and legal professionalism.

LegalEase empowers entrepreneurs, professionals, and individuals to generate reliable legal content confidently—without needing a legal background.

Scenarios:

Scenario 1: A startup founder enters details about a new hire and selects "Employment Contract" as the document type. LegalEase generates a comprehensive agreement with clauses for roles, responsibilities, compensation, and confidentiality. The user customizes the final wording, adds their company logo, and downloads the document as a branded PDF.

Scenario 2: A freelancer needs a Non-Disclosure Agreement for a potential client. By specifying the parties, scope of confidentiality, and effective date, they instantly receive a structured and legally sound NDA.

Scenario 3: A landlord uses LegalEase to draft a residential lease agreement. By inputting the property address, tenant details, and lease terms, the platform creates a formatted document with editable sections and a downloadable version in .DOCX format.

Pre-requisites

1. FastAPI Framework Knowledge: FastAPI Documentation
2. Google Gemini API Familiarity: Google Generative AI Documentation
3. Streamlit for Frontend: Streamlit Documentation
4. Python Programming Proficiency: Python Official Docs
5. Environment Setup: pip install fastapi uvicorn streamlit python-docx fpdf Pillow requests google-generativeai python-dotenv

---

Project Workflow

Activity 1: Model Selection and System Architecture

- **1.1: Research and select the Model**: Gemini 1.5 Pro model from Google Generative AI for legal document generation.
- **1.2: Define architecture:**
    - Frontend: Streamlit for user input and document download.
    - Backend: FastAPI (main.py, routes.py) to handle API requests.
    - AI Integration: Gemini 1.5 API used inside ai_core/gemini_generator.py.
- **1.3: Set up development environment with .env file containing the Gemini API key.**

Activity 2: Core Functionalities Development

- **2.1: Implement core generation features:**
    - Legal document creation
    - Editable text preview
    - Format & export as .txt, .docx, and .pdf
- **2.2: Build FastAPI backend:**
    - Endpoint /generate to process POST requests
    - Accepts document_type, parties, terms, dates
    - Returns generated text from Gemini

Activity 3: main.py & API Logic

- **3.1: Create and configure main.py:**
  - Initialize FastAPI app
  - Route / for home
  - Include router from routes.py
- **3.2: Handle document generation logic in routes.py:**
  - Define Pydantic model DocumentRequest
  - Call GeminiDocumentGenerator.generate_document(...)

Activity 4: Frontend with Streamlit

- **4.1: Design user-friendly Streamlit interface:**
  - Input fields for all required data
  - Responsive layout with title and logo
- **4.2: Dynamic document generation preview:**
  - Show styled editable output
  - Option to export as .txt, .docx, or .pdf

Activity 5: Deployment

- **5.1: Prepare deployment:**
  - Install production-ready dependencies
  - Set up Procfile, requirements.txt, or Dockerfile if needed
- **5.2: Deploy application:**
  - Options: Render, Railway, Deta, Streamlit Community Cloud (frontend), Fly.io or VPS for backend

---

Activity 6: Conclusion

- **Finalize and test:**
  - Ensure end-to-end functionality
  - Test Gemini integration under different input combinations

MILESTONE 1: Model Selection and Architecture – LegalEase

In this milestone, we focus on selecting the optimal generative AI model for creating personalized and accurate legal documents using Google's Gemini model via the Google Generative AI SDK. We also define the system architecture and set up the development environment to support document generation, editing, and exporting in various formats (.docx, .pdf, .txt).

Activity 1.1: Research and Select the Appropriate Generative AI Model

1. **Understand the Project Requirements**
    - LegalEase must generate structured legal documents (e.g., contracts, NDAs, agreements) based on user inputs such as:
        - Document type
        - Parties involved
        - Terms and conditions
        - Effective dates
2. **Explore Gemini Model Documentation**
    - Visit Google Generative AI Documentation to evaluate available Gemini models.
    - Review capabilities of gemini-1.5-pro, especially:
        - Long context window (up to 1 million tokens)
        - Structured prompting with multi-part content
        - Fast generation speed and legal-domain compatibility
3. **Evaluate Model Performance**
    - Benchmarks indicate that gemini-1.5-pro is capable of handling complex formatting, reasoning, and structured text generation, making it suitable for legal content generation.
4. **Select the Optimal Model**

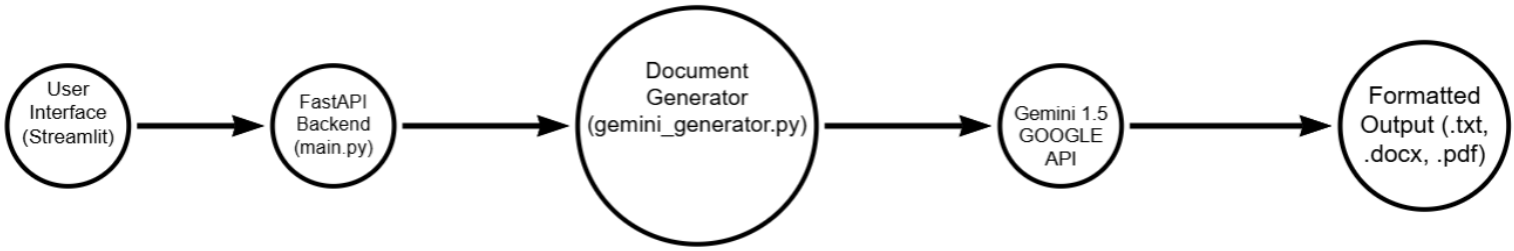
    Selected: gemini-1.5-pro

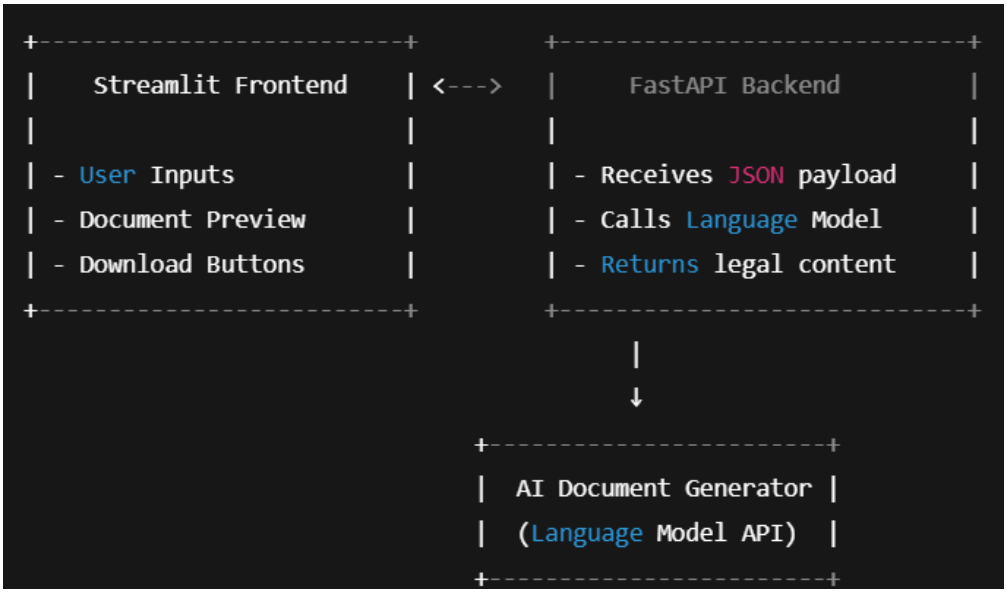
        - Best suited for drafting formal documents with clear sections and legal jargon.
        - Capable of understanding and generating clauses based on user inputs dynamically.

Activity 1.2: Define the Architecture of the Application

1. Architectural Diagram

- Components:
  - **Frontend**: Streamlit for user interaction (document input, preview, editing, downloads)
  - **Backend**: FastAPI app (routes input to Gemini model and returns generated content)
  - **AI Core**: Gemini integration for content generation
  - **Formatting Modules**: DOCX, PDF, TXT generators





2. Frontend Functionality

- Users provide inputs via:
  - TextInput for document type and dates
  - TextArea for parties and terms
- Buttons:
  - "Generate Document"
  - "Edit Document"
  - "Download as TXT / DOCX / PDF"
- Styled HTML preview of the generated legal document

3. Backend Responsibilities

- Expose POST /generate endpoint (via FastAPI)
- Accept JSON input (document_type, parties, terms, dates)
- Route to GeminiDocumentGenerator
- Return structured document text

4. AI Integration Points

- GeminiDocumentGenerator forms a structured prompt and sends it to Gemini via generate_content
- Receives generated text and returns to FastAPI
- Frontend displays and allows editing or exporting

Activity 1.3: Set Up the Development Environment

1. **Install Python and Pip**
   Ensure Python 3.10+ is installed along with pip for dependency management.
2. **Create a Virtual Environment**
3. **Install FastAPI and Uvicorn (Backend Server)**

pip install fastapi uvicorn

4. **Install Streamlit (Frontend UI)**

pip install streamlit

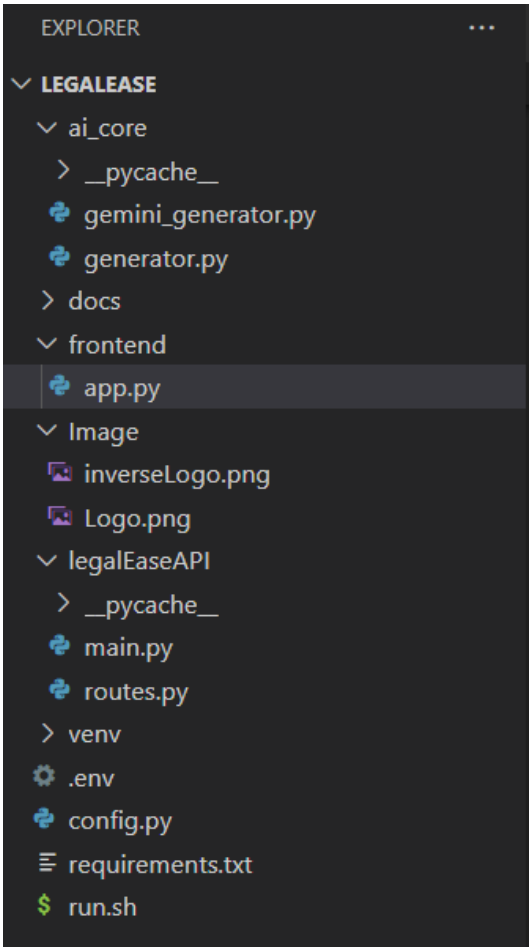
5. **Install Google Generative AI SDK**

pip install google-generativeai

6. **Install Additional Libraries**

pip install python-docx fpdf Pillow requests python-dotenv

7. **Set Up Application Structure**



MILESTONE 2: Core Functionalities Development

Activity 2.1: Develop the core functionalities

1. Legal Document Generation

Functionality: Generate legal documents (e.g., contracts, agreements) based on user inputs (document type, parties, terms, and dates).

```python
# routes.py
@router.post("/generate")
def generate_legal_document(request: DocumentRequest):
    response = gemini_generator.generate_document(
        request.document_type,
        request.parties,
        request.terms,
        request.dates
    )
    return {"document": response}
```

```python
# gemini_generator.py
def generate_document(self, document_type, parties, terms, dates):
    prompt = (
        f"Generate a comprehensive legal document titled '{document_type}'\n"
        f"Involved parties: {parties}\n"
        f"Effective Date: {dates}\n"
        f"Terms and conditions: {terms}\n"
        f"Ensure formal legal structure with multiple sections and legal clauses."
    )
    response = self.model.generate_content(prompt)
    return response.text
```

2. Document Output Formatting

Functionality: Convert raw AI-generated text into structured .docx, .pdf, .txt files with legal formatting, logos, tables, and footers.

- format_docx(text, doc_type)
- format_pdf(text, doc_type)

- format_html_preview(text)

These use:

- **docx** for Microsoft Word formatting (includes logo, Times New Roman font, terms as table, footer).
- **fpdf** for clean, branded PDF documents with headers/footers.
- **HTML Preview** with Streamlit styling.

---

3. User Interaction & Recommendations

Functionality: Accept user preferences through the Streamlit interface and reflect them in the AI-generated legal content.

```
# app.py - Streamlit UI
document_type = st.text_input("Document Type")
parties = st.text_area("Parties Involved")
terms = st.text_area("Terms & Conditions")
dates = st.text_input("Effective Date")

# Send to backend
response = requests.post("http://localhost:8000/generate", json={
    "document_type": document_type,
    "parties": parties,
    "terms": terms,
    "dates": dates
})
```

4. Editable Document & Download

Functionality: Provides editable text area, allows download in .txt, .docx, .pdf formats.

```
# app.py
if st.session_state.get("show_edit"):
    edited_text = st.text_area("Edit Document Below:", generated_text, height=300)
    st.session_state.generated_text = edited_text

# Downloads
st.download_button("📄 Download as .TXT", data=generated_text, ...)
st.download_button("📝 Download as .DOCX", data=format_docx(...), ...)
st.download_button("📕 Download as .PDF", data=format_pdf(...), ...)
```

Activity 2.2: Implement the FastAPI Backend

1. Define Routes in FastAPI (Instead of Flask)

```
# main.py
app.include_router(router)

# routes.py
@router.post("/generate")  # Route for generating legal document
```

2. Process User Input

Handled via Pydantic & Streamlit Forms:

```
# routes.py
class DocumentRequest(BaseModel):
    document_type: str
    parties: str
    terms: str
    dates: str
```

```
# app.py
# Streamlit collects input → FastAPI handles it
response = requests.post("http://localhost:8000/generate", json={...})
```

3. **Integrate Groq (Gemini) API Calls**

```
# gemini_generator.py
genai.configure(api_key=os.getenv("GEMINI_API_KEY"))

self.model = genai.GenerativeModel(model_name)
response = self.model.generate_content(prompt)
```

MILESTONE 3: API Logic Integration

In **Milestone 3**, the focus shifts to designing and integrating the backend API logic that powers document generation using the Gemini 1.5 Pro model. This milestone ensures a functional bridge between frontend inputs and the AI-powered document generation, which is the core of the LegalEase project.

Activity 3.1: Create and Configure main.py

This step involves initializing the FastAPI application, setting up the base endpoint for health-checks, and including modular API routes.

Steps:

- Initialize the FastAPI app
- Define a root (/) endpoint to verify the service is running
- Include routes from the routes.py module

```python
from fastapi import FastAPI
from legalEaseAPI.routes import router

app = FastAPI(title="LegalEase - AI Legal Document Generator")

# Include API routes
app.include_router(router)

# Root endpoint
@app.get("/")
def home():
    return {"message": "Welcome to LegalEase AI Legal Document Generator API"}

# Run FastAPI if executed directly
if __name__ == "__main__":
    import uvicorn
    uvicorn.run(app, host="0.0.0.0", port=8000)
```

Activity 3.2: Handle Document Generation Logic in routes.py

This task handles user inputs from the frontend and sends them to the Gemini model for document generation.

Steps:

- Define a Pydantic DocumentRequest model to validate incoming data.
- Initialize the GeminiDocumentGenerator from ai_core.gemini_generator.
- Create a POST endpoint (/generate) to accept user input and return generated legal content.

```python
from fastapi import APIRouter
from pydantic import BaseModel
from ai_core.gemini_generator import GeminiDocumentGenerator

router = APIRouter()
gemini_generator = GeminiDocumentGenerator()

class DocumentRequest(BaseModel):
    document_type: str
    parties: str
    terms: str
    dates: str

@router.post("/generate")
def generate_legal_document(request: DocumentRequest):
    response = gemini_generator.generate_document(
        request.document_type,
        request.parties,
        request.terms,
        request.dates
    )
    return {"document": response}
```

MILESTONE 4: Frontend Development – LegalEase

Milestone 4 focuses on creating a responsive and user-centric web interface for **LegalEase** using **Streamlit**. This milestone enables users to easily input legal details, generate AI-powered documents, preview the results, make inline edits, and download documents in multiple formats (TXT, DOCX, PDF) with rich formatting and embedded branding.

Activity 4.1: Designing and Developing the User Interface

Step 1: Streamlit Page Configuration and Layout Setup

- Configured Streamlit app with:

```
st.set_page_config(page_title="LegalEase", layout="centered")
```

- Added a three-column layout using st.columns to center-align the company logo

```
col1, col2, col3 = st.columns([1, 2, 1])
with col2:
    st.image(WEB_LOGO_PATH, use_container_width=True)
```

Step 2: Header and Title

- Displayed a subtle center-aligned title under the logo:

```
st.markdown(
    "<h2 style='text-align: center;'>AI Legal Document Generator</h2>",
    unsafe_allow_html=True
)
```

Step 3: User Input Interface

- Created intuitive inputs for:
  - document_type: type of legal document
  - parties: involved stakeholders
  - terms: semicolon-separated clauses
  - dates: effective date

```
document_type = st.text_input("Document Type")
parties = st.text_area("Parties Involved")
terms = st.text_area("Terms & Conditions (Use semicolons for bullet points)")
dates = st.text_input("Effective Date")
```

Activity 4.2: Creating Dynamic Previews and Export Options

Step 1: Generate Document with Backend AI

- On clicking "Generate Document", a POST request is sent to the FastAPI backend:

```
response = requests.post("http://localhost:8000/generate", json={...})
```

- AI-generated content is sanitized and displayed dynamically:

```
generated_text = sanitize_text(response.json()["document"])
```

Step 2: HTML Preview Rendering

- The output is previewed in an elegant dark-themed scrollable card with semantic HTML:

```
styled_html = format_html_preview(generated_text)
st.markdown(f"<div style='...'>{styled_html}</div>", unsafe_allow_html=True)
```

Step 3: Editable Document Preview

- The user can click "Edit Document" and modify the generated content using a text area:

```
edited_text = st.text_area("Edit Document Below:", generated_text, height=300)
```

Step 4: Multi-Format Download Options

- Users can download the document as:
  - .txt (Plain Text)
  - .docx (Formatted Word Document with logo, headings, and table)
  - .pdf (Branded PDF with logo and footer)

```
st.download_button("📄 Download as .TXT", data=generated_text, file_name=f"{document_type.replace(' ', '_').lower(
st.download_button("📝 Download as .DOCX", data=format_docx(generated_text, document_type), file_name=f"{document_
st.download_button("📕 Download as .PDF", data=format_pdf(generated_text, document_type), file_name=f"{document_ty
```

Core Functions & Utilities

sanitize_text(text)

- Removes special characters and typographic quotes to ensure clean formatting.

format_docx(text, doc_type)

- Uses python-docx to:
  - Embed logo
  - Add titles, paragraph formatting
  - Auto-generate a Terms table from semicolon-separated input

format_pdf(text, doc_type)

- Utilizes FPDF with custom header/footer:
  - Center-aligned logo
  - Bold headings for sections
  - Bullet-style terms

format_html_preview(text)

- Converts output to stylized HTML blocks for inline display.

MILESTONE 5: Deployment

In Milestone 5, the focus is on deploying the **LegalEase** application locally using **FastAPI** as the backend and **Streamlit** as the frontend. This milestone involves setting up a virtual environment, configuring dependencies, and verifying that all modules—from the Gemini API integration to document generation and downloads—work seamlessly in a real-world local environment. This ensures your system is production-ready and paves the way for cloud deployment in future stages.

Activity 5.1: Preparing the Application for Local Deployment

Step 1: Set Up a Virtual Environment

To ensure environment isolation and consistent dependency management, create and activate a virtual environment:

```
PS F:\VS CODE\python\LegalEase> venv\scripts\activate
(venv) PS F:\VS CODE\python\LegalEase>
```

Step 2: Install Dependencies

Ensure that the backend (FastAPI) and frontend (Streamlit) dependencies are installed from a requirements.txt:

pip install -r requirements.txt

requirements.txt

```
aiohappyeyeballs==2.6.1
aiohttp==3.11.16
aiosignal==1.3.2
altair==5.5.0
annotated-types==0.7.0
anyio==4.9.0
asgiref==3.8.1
attrs==25.3.0
backoff==2.2.1
bcrypt==4.3.0
blinker==1.9.0
build==1.2.2.post1
cachetools==5.5.2
certifi==2025.1.31
charset-normalizer==3.4.1
chroma-hnswlib==0.7.6
chromadb==1.0.0
click==8.1.8
colorama==0.4.6
coloredlogs==15.0.1
dataclasses-json==0.6.7
```

Step 3: Configure Environment Variables

Create a .env file to securely store your **Gemini API key**

Activity 5.2: Testing and Verifying Local Deployment

Step 1: Launch the FastAPI Backend

Start the backend server using **Uvicorn**:

```
(venv) PS F:\VS CODE\python\LegalEase> uvicorn legalEaseAPI.main:app --reload
INFO:     Will watch for changes in these directories: ['F:\\VS CODE\\python\\Leg
alEase']
INFO:     Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)INFO:
  Started reloader process [15168] using WatchFiles
INFO:     Started server process [17360]
INFO:     Waiting for application startup.
INFO:     Application startup complete.
```

Step 2: Launch the Streamlit Frontend

Run the app.py file to start the **LegalEase** UI:

```
(venv) PS F:\VS CODE\python\LegalEase> streamlit run frontend/app.py

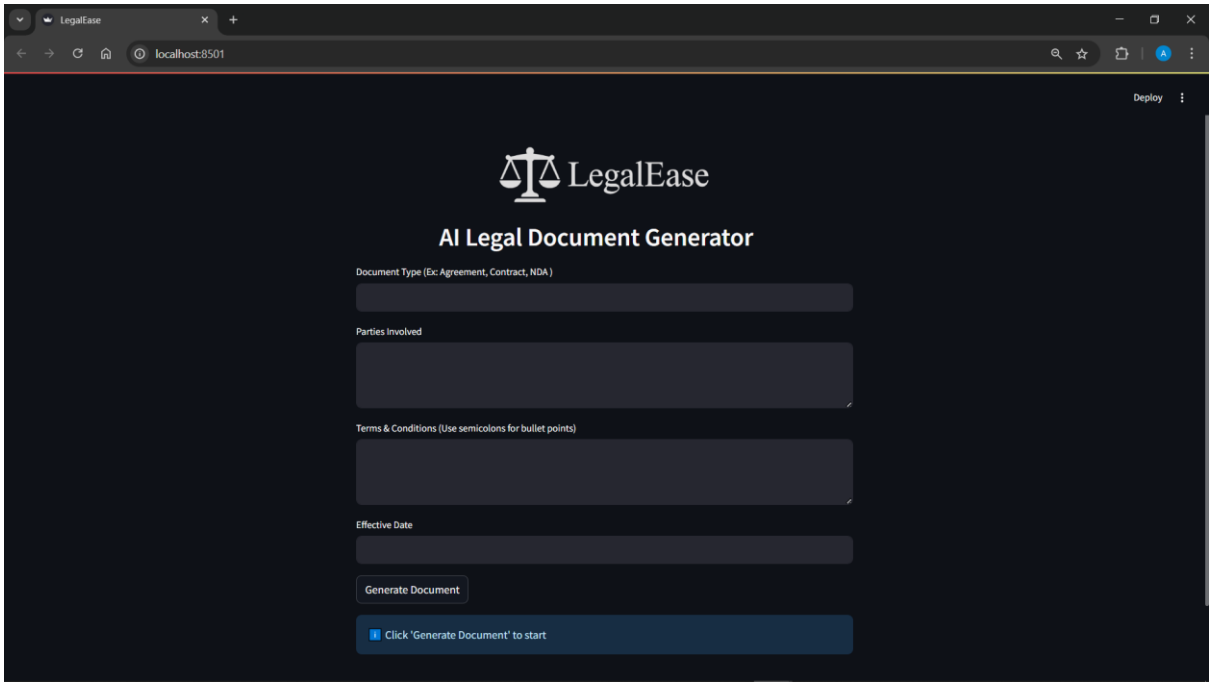
  You can now view your Streamlit app in your browser.

  Local URL: http://localhost:8501
  Network URL: http://192.168.1.3:8501
```

Step 3: Test All Functionalities

- Input document type, parties, terms, and date.
- Press **Generate Document** to get AI-generated legal content from Gemini.
- Use built-in options to:
  - Preview in dark mode HTML
  - Edit the text
  - Download in .txt, .docx, and .pdf formats
- Check for:
  - Proper layout with logo and footer
  - Clean formatting of bullet points
  - Correct embedding of terms in the document

Exploring LegalEase



1. Document Type

Purpose: Type of legal document to generate.
Examples:

- Agreement
- Contract
- NDA (Non-Disclosure Agreement)
- Lease Agreement
- Employment Offer Letter

Example input: Freelance Work Contract

2. Parties Involved

Purpose: Names and roles of the individuals or entities involved in the document.
Examples:

- John Doe (Freelancer), ABC Corp (Client)
- Alice Smith (Tenant), XYZ Realty (Landlord)

Example input: Jane Doe (Service Provider), TechNova Inc. (Client)

3. Terms & Conditions (Use semicolons for bullet points)

Purpose: Specific clauses or rules that define the agreement. Separate each bullet point using a semicolon.

Examples:

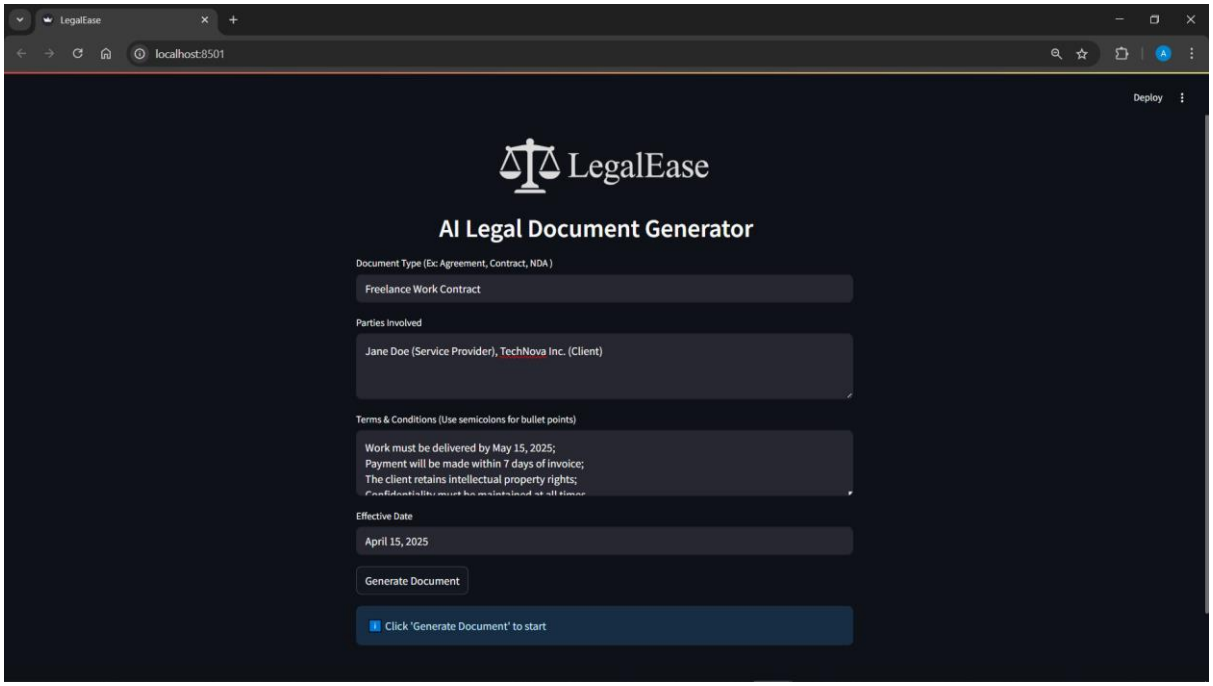
- Payment to be made within 30 days of invoice;
- The provider agrees to deliver work by the agreed deadline;
- Confidentiality must be maintained at all times;
- Either party may terminate with 15 days notice

4. Effective Date

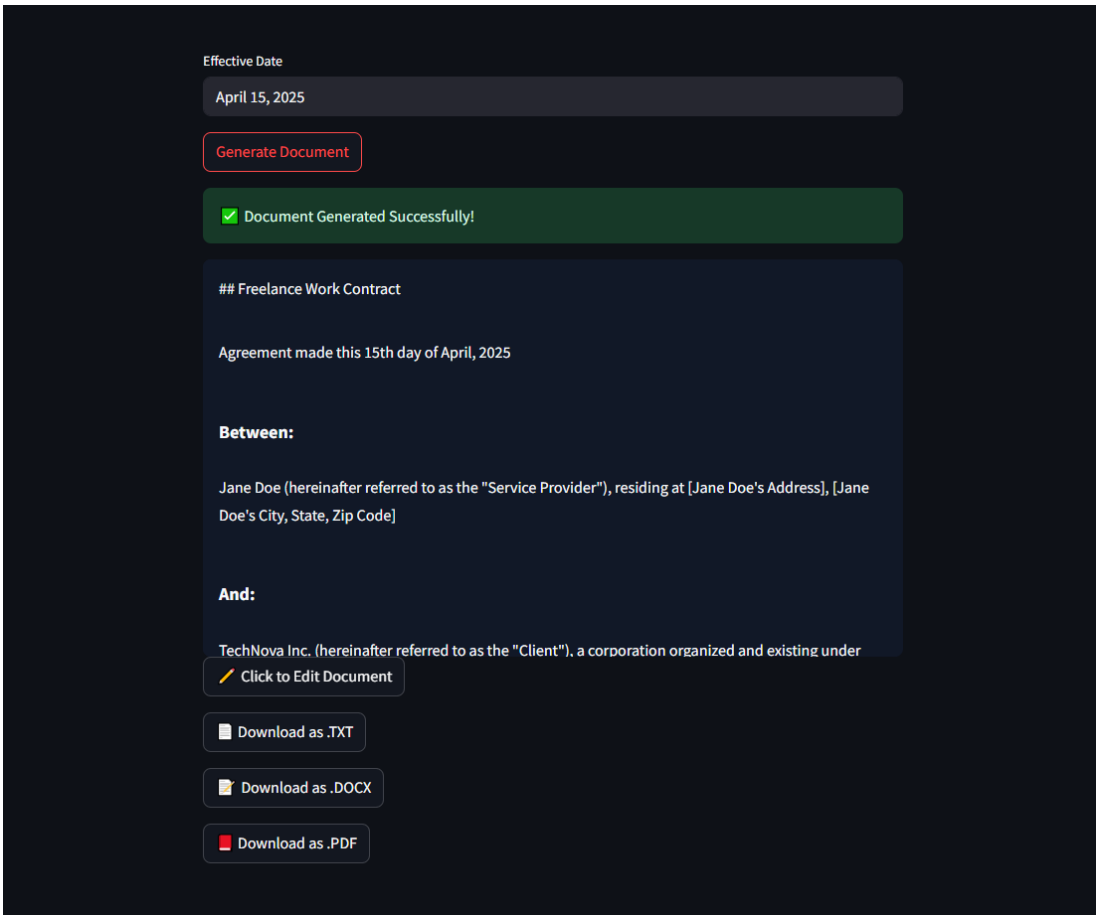
Purpose: The date when the agreement becomes legally valid.

Examples:

- April 10, 2025
- 10/04/2025
- April 15, 2025

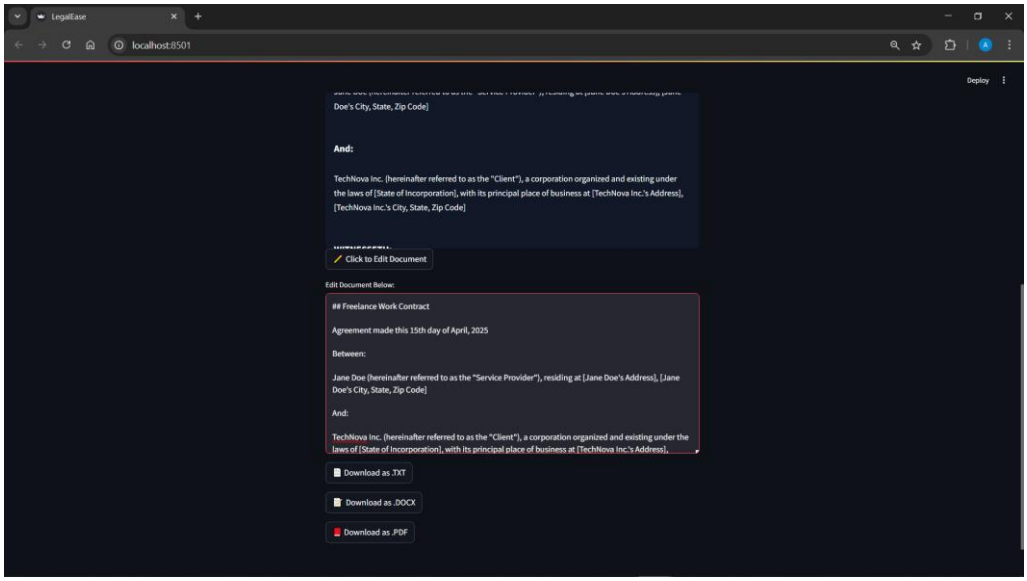


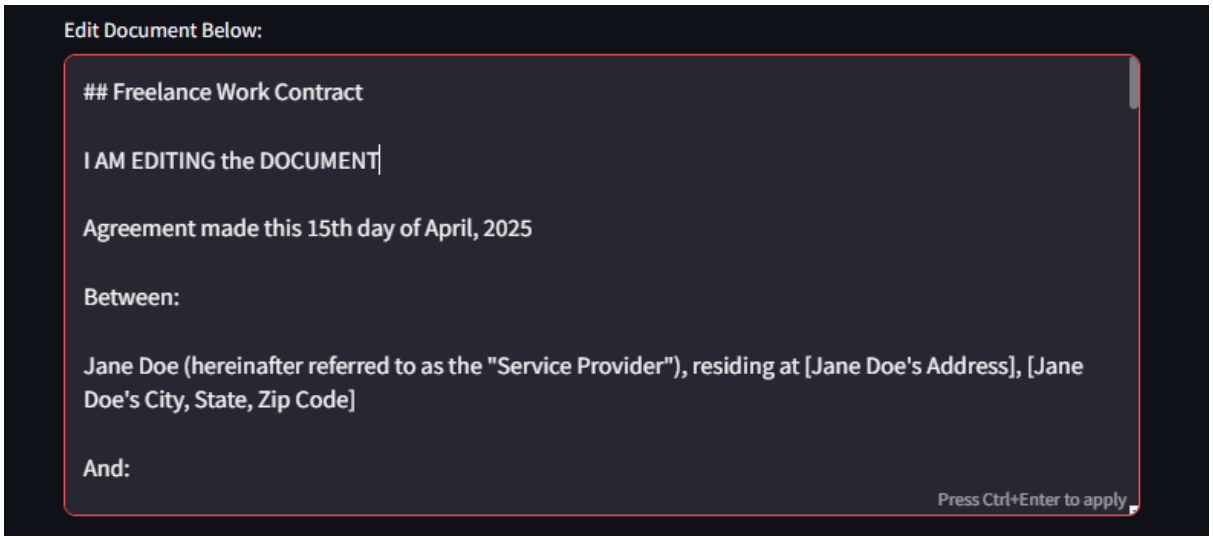
Document Generated



Editing the Document

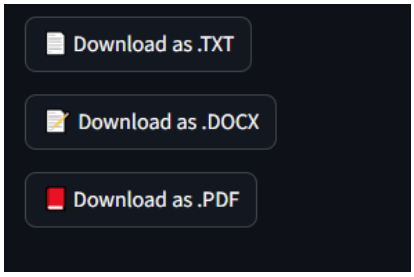
If the user wants to edit the document, they can do so by clicking on the " Click to Edit Document" Botton. A new window opens up where the document can be edited.



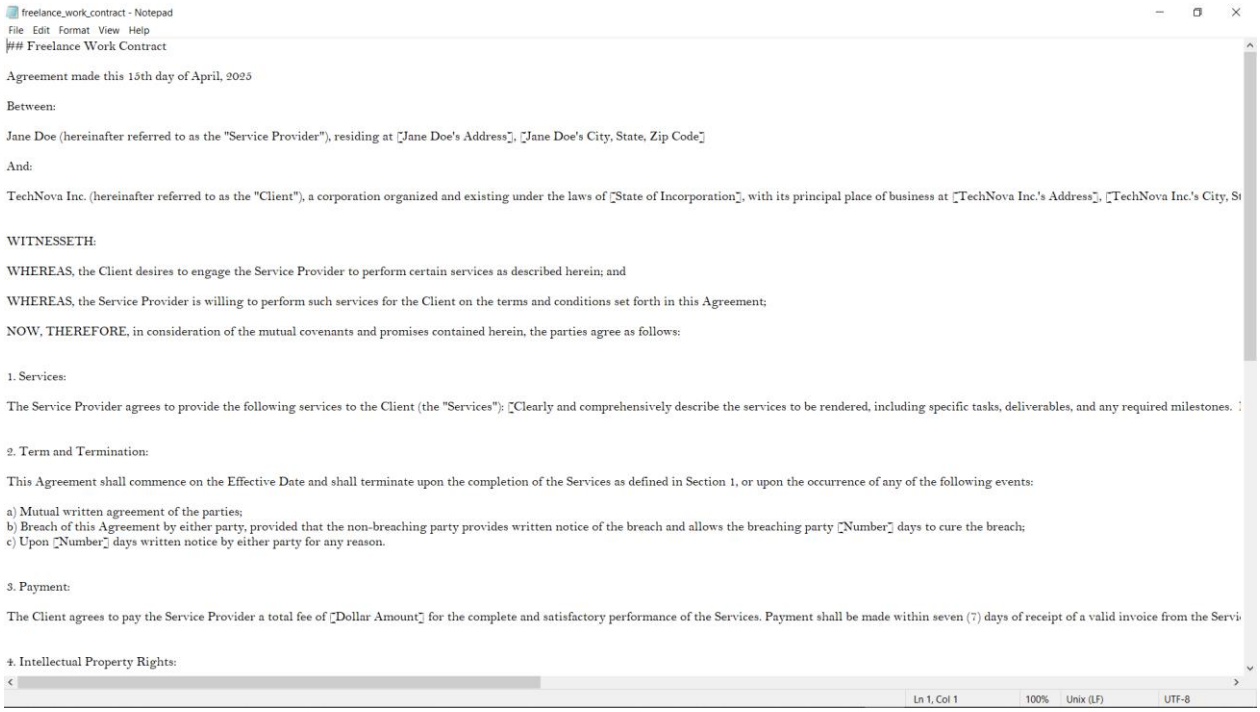


Downloading/ Saving the Generated Document

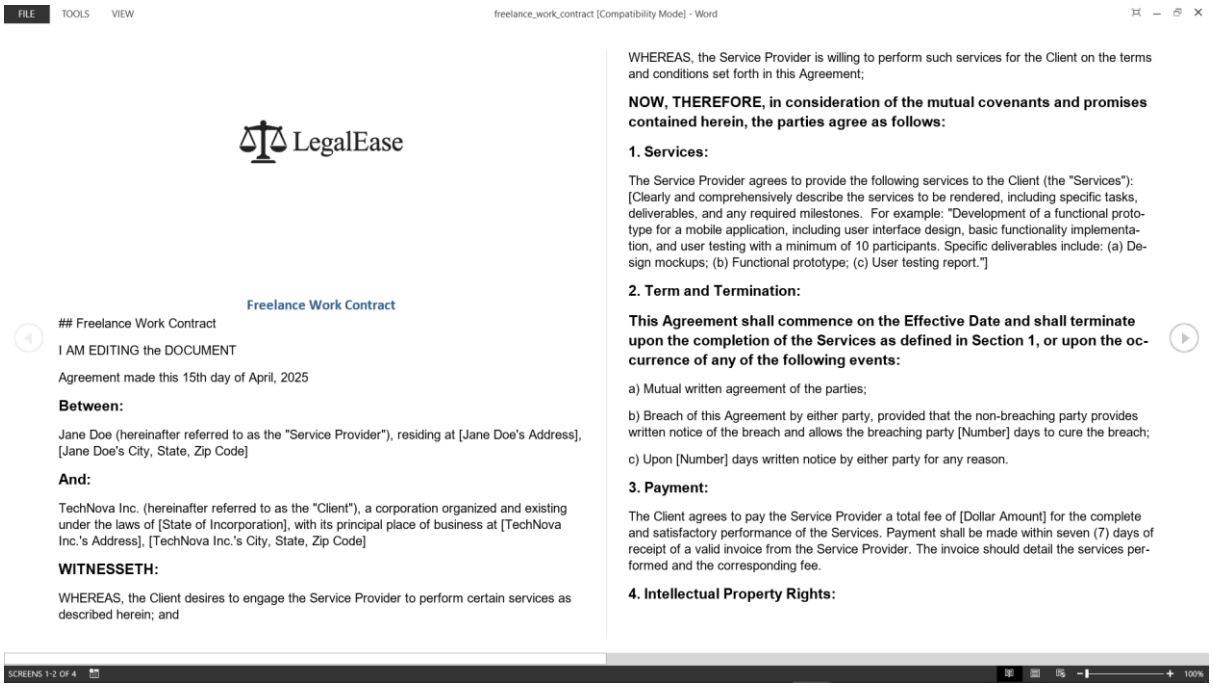
We can Download and save the documents in any of the following formats.

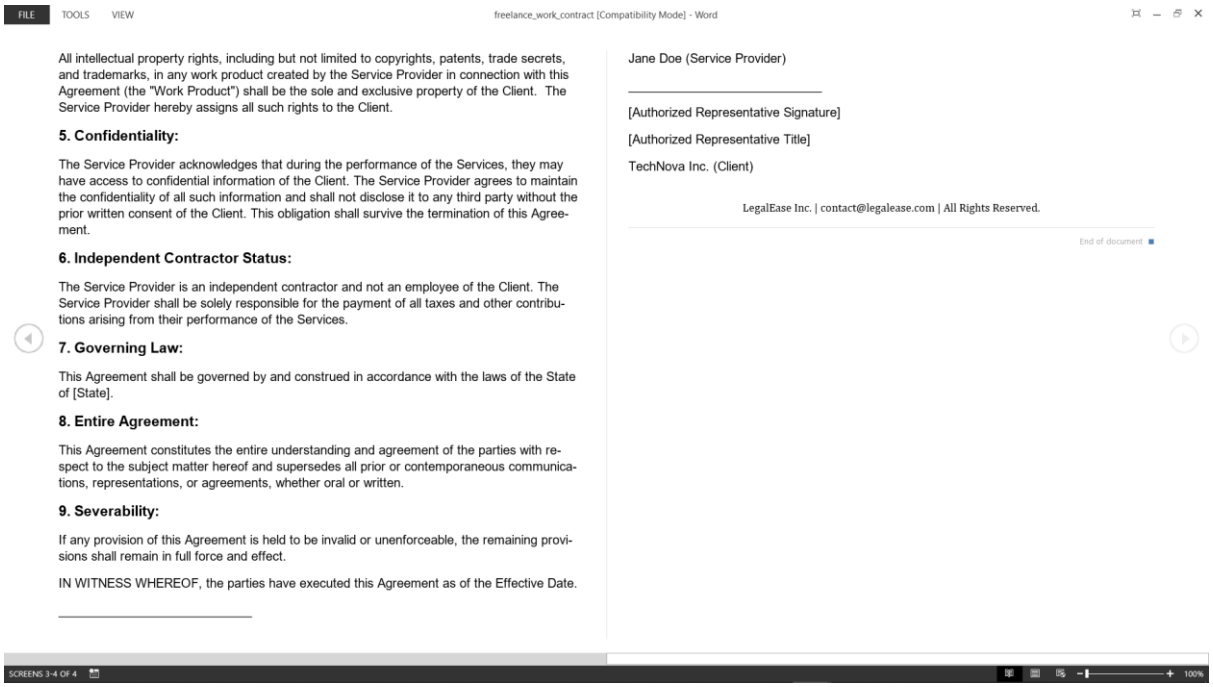


- **.txt**

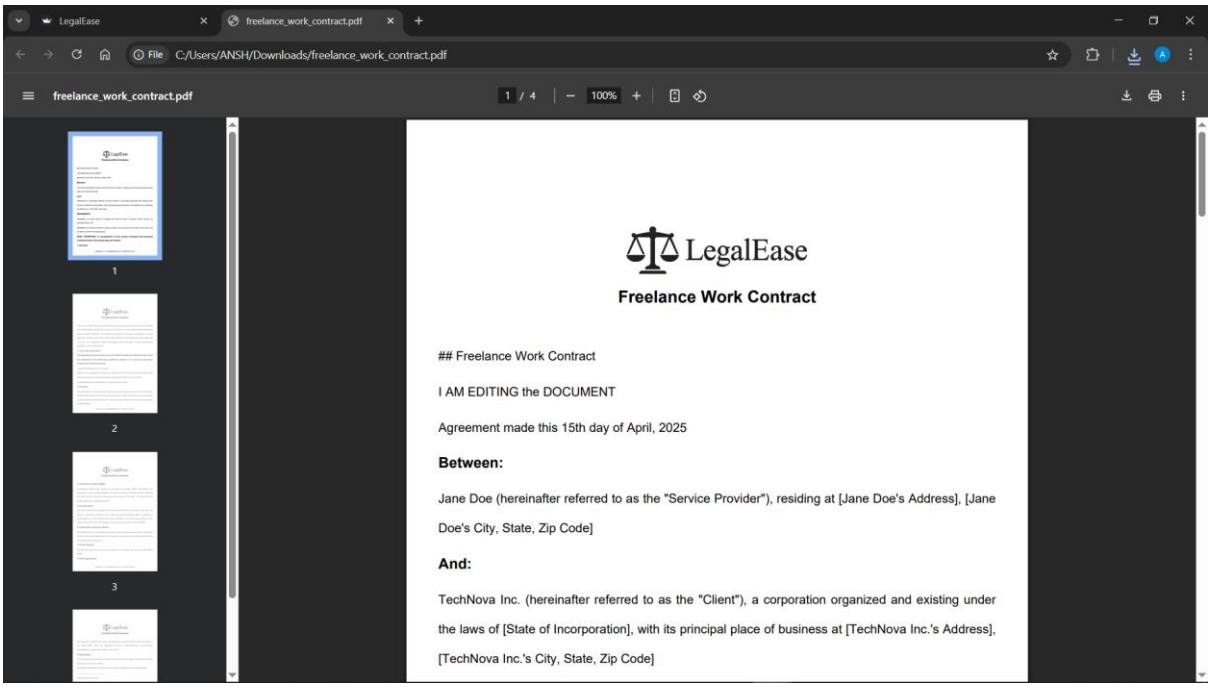


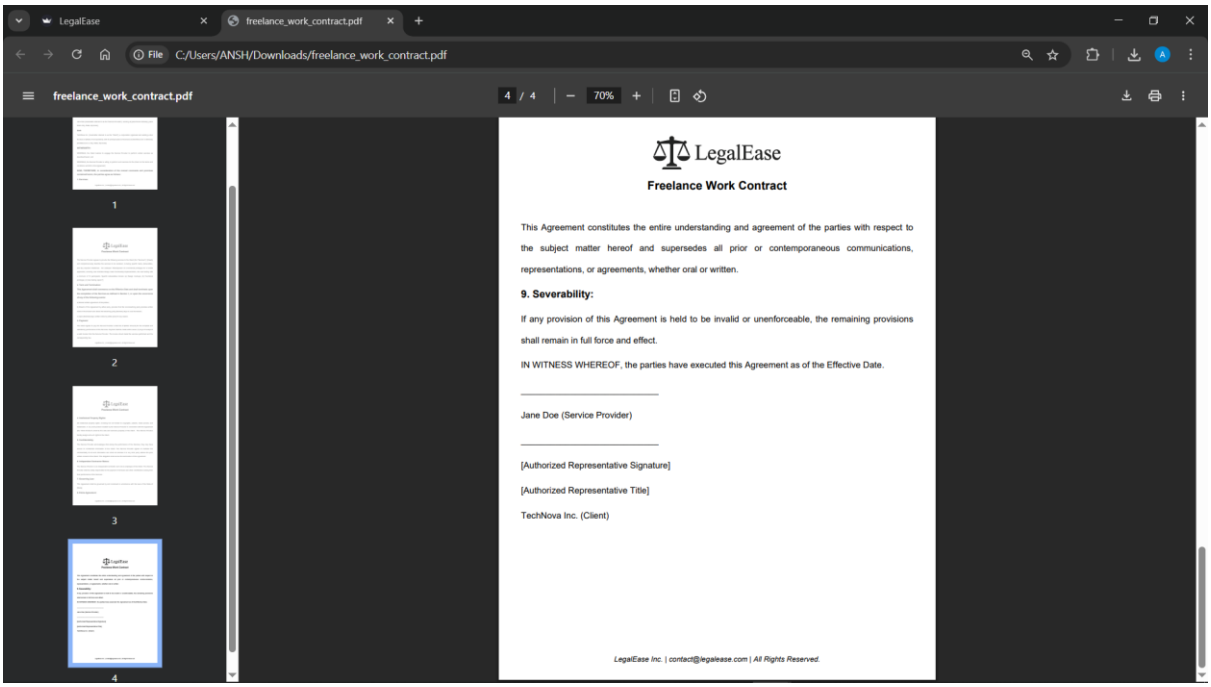
- **.docx**
  The document provided inculcates the use of the logo (front page) as well as the footer in the last page





- **.pdf**
  This format provided the user with a well formatted and ready to use legal document with Logo and Footer on all the pages.




LegalEase
freelance_work_contract.pdf
File C:/Users/ANSH/Downloads/freelance_work_contract.pdf
freelance_work_contract.pdf
4 / 4
70%
1
2
3
4
LegalEase
Freelance Work Contract
This Agreement constitutes the entire understanding and agreement of the parties with respect to the subject matter hereof and supersedes all prior or contemporaneous communications, representations, or agreements, whether oral or written.
9. Severability:
If any provision of this Agreement is held to be invalid or unenforceable, the remaining provisions shall remain in full force and effect.
IN WITNESS WHEREOF, the parties have executed this Agreement as of the Effective Date.
Jane Doe (Service Provider)
[Authorized Representative Signature]
[Authorized Representative Title]
TechNova Inc. (Client)
LegalEase Inc. | contact@legalease.com | All Rights Reserved.

Conclusion

LegalEase represents a transformative step in bridging the accessibility gap in legal services through the power of artificial intelligence. This project demonstrates how emerging technologies can simplify complex legal language, providing users—regardless of legal background—with a clear, concise understanding of legal documents. By integrating advanced natural language processing (NLP) models with user-centric design, LegalEase empowers individuals to navigate legal content with confidence, thereby promoting inclusivity and informed decision-making.

The platform's ability to analyze and simplify legal text, highlight key clauses, and provide plain-language summaries enhances comprehension and reduces dependency on costly legal consultations for basic understanding. This democratization of legal knowledge is critical in a world where legal literacy remains low, especially among marginalized populations. Furthermore, LegalEase supports multilingual capabilities, ensuring broader reach and utility across different regions and demographics.

From a technical standpoint, LegalEase showcases efficient integration of backend AI models, including document parsing, summarization algorithms, and user-friendly web interfaces. The project's architecture is designed to ensure scalability, data privacy, and real-time responsiveness—key elements for any modern SaaS solution. Through iterative development, testing, and feedback, the system has been refined for optimal performance and user satisfaction.

In conclusion, LegalEase is more than a technological tool—it is a catalyst for legal empowerment. By fostering transparency, accessibility, and understanding, it upholds the principle that legal information should be a public good, not a privileged resource. Future enhancements could include deeper contract analysis, integration with legal databases, and personalized recommendations. With continued innovation and support, LegalEase has the potential to revolutionize how people interact with the law, making justice more approachable for all.